FUNCTIONS Task1: Create a Python program that converts between different units of measurement

```
def convert_units(conversion_type, value):
    """
    Converts units based on the selected conversion type.

    Parameters:
    conversion_type (int): The option selected by the user for conversion.
    value (float): The number to convert.

    Returns:
    float: The converted value.
    """

    if conversion_type == 1:
        return value * 3.28084  # meters to feet
    elif conversion_type == 2:
        return value / 3.28084  # feet to meters
    elif conversion_type == 3:
        return value * 2.20462  # kg to pounds
    elif conversion_type == 4:
        return value / 2.20462  # pounds to kg
    elif conversion_type == 5:
        return value * 0.264172  # liters to gallons
    elif conversion_type == 6:
        return value / 0.264172  # gallons to liters
    else:
        return None


try:
    print("Choose conversion type:")
    print("1. Meters to Feet")
    print("2. Feet to Meters")
    print("3. Kilograms to Pounds")
    print("4. Pounds to Kilograms")
    print("5. Liters to Gallons")
    print("6. Gallons to Liters")

    choice = int(input("Enter option number: "))
    value = float(input("Enter value to convert: "))

    result = convert_units(choice, value)

    if result is None:
        print("Invalid conversion type.")
    else:
        print("Converted value:", result)

except ValueError:
    print("Error: Please enter a valid number.")
```

```
Choose conversion type:
1. Meters to Feet
2. Feet to Meters
3. Kilograms to Pounds
4. Pounds to Kilograms
5. Liters to Gallons
6. Gallons to Liters
Enter option number: 3
Enter value to convert: 10
Converted value: 22.0462
```

Task2: Create a Python program that performs various mathematical operations on a list of numbers.

```
def find_sum(numbers):
    """Calculates the sum of a list of numbers."""
    return sum(numbers)

def find_average(numbers):
    """Calculates the average of a list of numbers."""
    return sum(numbers) / len(numbers)

def find_maximum(numbers):
```

```
    """Finds the maximum value in a list of numbers."""
    return max(numbers)

def find_minimum(numbers):
    """Finds the minimum value in a list of numbers."""
    return min(numbers)

def math_operations_program():
    print("1. Sum\n2. Average\n3. Maximum\n4. Minimum")

    try:
        choice = int(input("Select an operation (1-4): "))

        raw_input = input("Enter numbers separated by spaces: ")
        numbers = [float(x) for x in raw_input.split()]

        if not numbers:
            print("Error: The list cannot be empty.")
            return

        if choice == 1:
            print(f"Result (Sum): {find_sum(numbers)}")
        elif choice == 2:
            print(f"Result (Average): {find_average(numbers)}")
        elif choice == 3:
            print(f"Result (Maximum): {find_maximum(numbers)}")
        elif choice == 4:
            print(f"Result (Minimum): {find_minimum(numbers)}")
        else:
            print("Invalid selection. Please choose 1, 2, 3, or 4.")

    except ValueError:
        print("Error: Please enter only numeric values (numbers).")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

math_operations_program()
```

```
1. Sum
2. Average
3. Maximum
4. Minimum
Select an operation (1-4): 1
Enter numbers separated by spaces: 2
Result (Sum): 2.0
```

LIST MANIPULATION

Write a Python function that extracts every other element from a list, starting from the first element

```
def extract_every_other(lst):
    return lst[::2]

numbers = [1, 2, 3, 4, 5, 6]
result = extract_every_other(numbers)
print(result)
```

```
[1, 3, 5]
```

Write a Python function that returns a sublist from a given list, starting from a specified index and ending at another specified index

```
def get_sublist(lst, start, end):
    return lst[start:end+1]

numbers = [1, 2, 3, 4, 5, 6]
result = get_sublist(numbers, 2, 4)
print(result)
```

```
[3, 4, 5]
```

Write a Python function that reverses a list using slicing

```
def reverse_list(lst):
    return lst[::-1]

numbers = [1, 2, 3, 4, 5]
result = reverse_list(numbers)
print(result)
```

```
[5, 4, 3, 2, 1]
```

Write a Python function that removes the first and last elements of a list and returns the resulting sublist.

```
def remove_first_last(lst):
    return lst[1:-1]

numbers = [1, 2, 3, 4, 5]
result = remove_first_last(numbers)
print(result)
```

```
[2, 3, 4]
```

Write a Python function that extracts the first n elements from a list

```
def get_first_n(lst, n):
    return lst[:n]

numbers = [1, 2, 3, 4, 5]
result = get_first_n(numbers, 3)
print(result)
```

```
[1, 2, 3]
```

Write a Python function that extracts the last n elements of a list using slicing

```
def get_last_n(lst, n):
    return lst[-n:]

numbers = [1, 2, 3, 4, 5]
result = get_last_n(numbers, 2)
print(result)
```

```
[4, 5]
```

Write a Python function that extracts a list of elements in reverse order starting from the second-to-last element and skipping one element in between

```
def reverse_skip(lst):
    return lst[-2::-2]

numbers = [1, 2, 3, 4, 5, 6]
result = reverse_skip(numbers)
print(result)
```

NESTED LIST

Write a Python function that takes a nested list and flattens it into a single list, where all the elements are in a single dimension

```
def flatten(lst):
    flat_list = []

    for sublist in lst:
        for item in sublist:
            flat_list.append(item)
```

```
    return flat_list

data = [[1, 2], [3, 4], [5]]
print(flatten(data))
```

```
[1, 2, 3, 4, 5]
```

Write a Python function that extracts a specific element from a nested list given its indices

```
def access_nested_element(lst, indices):
    return lst[indices[0]][indices[1]]
lst = [[1,2,3],[4,5,6],[7,8,9]]
indices = [1,2]

print(access_nested_element(lst, indices))
```

```
6
```

Write a Python function that calculates the sum of all the numbers in a nested list (regardless of depth)

```
def sum_nested(lst):
    total = 0

    for item in lst:
        if isinstance(item, list):
            total = total + sum_nested(item)
        else:
            total = total + item

    return total

data = [[1,2],[3,[4,5]],6]
print(sum_nested(data))
```

```
21
```

Write a Python function that removes all occurrences of a specific element from a nested list

```
def remove_element(lst, elem):
    new_list = []

    for sublist in lst:
        temp = []
        for item in sublist:
            if item != elem:
                temp.append(item)
        new_list.append(temp)

    return new_list

lst = [[1,2],[3,2],[4,5]]
print(remove_element(lst,2))
```

```
[[1], [3], [4, 5]]
```

Write a Python function that finds the maximum element in a nested list (regardless of depth)

```
def find_max(lst):
    maximum = float("-inf")

    for item in lst:
        if isinstance(item, list):
            value = find_max(item)
            if value > maximum:
                maximum = value
        else:
            if item > maximum:
```

```
                maximum = item

    return maximum

data = [[1,2],[3,[4,5]],6]
print(find_max(data))
```

```
6
```

Write a Python function that counts how many times a specific element appears in a nested list.

```
def count_occurrences(lst, elem):
    count = 0

    for item in lst:
        if isinstance(item, list):
            count = count + count_occurrences(item, elem)
        else:
            if item == elem:
                count = count + 1

    return count
lst = [[1,2],[2,3],[2,4]]
print(count_occurrences(lst,2))
```

```
3
```

Write a Python function that flattens a list of lists of lists into a single list, regardless of the depth

```
def deep_flatten(lst):
    result = []

    for item in lst:
        if isinstance(item, list):
            result = result + deep_flatten(item)
        else:
            result.append(item)

    return result

data = [[[1,2],[3,4]],[[5,6],[7,8]]]
print(deep_flatten(data))
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Write a Python function that calculates the average of all elements in a nested list.

```
def average_nested(lst):

    def helper(lst):
        total = 0
        count = 0

        for item in lst:
            if isinstance(item, list):
                t, c = helper(item)
                total = total + t
                count = count + c
            else:
                total = total + item
                count = count + 1

        return total, count

    total, count = helper(lst)
    return total / count

data = [[1,2],[3,4],[5,6]]
print(average_nested(data))
```

```
3.5
```

Experiment: How Fast is Numpy?

Task 1: Compare element-wise addition between Python lists and NumPy arrays for 1,000,000 elements.

```
import time
import numpy as np
```

```
list1 = list(range(1000000))
list2 = list(range(1000000))

start = time.time()
result_list = [list1[i] + list2[i] for i in range(len(list1))]

end = time.time()

print("Time taken using Python lists:", end - start)

array1 = np.array(list1)
array2 = np.array(list2)

start = time.time()
result_array = array1 + array2

end = time.time()

print("Time taken using NumPy arrays:", end - start)
```

```
Time taken using Python lists: 0.10836148262023926
Time taken using NumPy arrays: 0.005545377731323242
```

Task 2: Compare element-wise multiplication for 1,000,000 elements.

```
start = time.time()
result_list = [list1[i] * list2[i] for i in range(len(list1))]
end = time.time()

print("Time taken using Python lists:", end - start)

start = time.time()
result_array = array1 * array2
end = time.time()

print("Time taken using NumPy arrays:", end - start)
```

```
Time taken using Python lists: 0.1253509521484375
Time taken using NumPy arrays: 0.004961729049682617
```

Task 3: Compare dot product performance for 1,000,000 elements.

```
start = time.time()

dot_product = 0
for i in range(len(list1)):
    dot_product += list1[i] * list2[i]

end = time.time()

print("Time taken using Python lists:", end - start)

start = time.time()
dot_product_np = np.dot(array1, array2)

end = time.time()

print("Time taken using NumPy arrays:", end - start)
```

```
Time taken using Python lists: 0.23021149635314941
Time taken using NumPy arrays: 0.0018188953399658203
```

Task 4: Compare matrix multiplication for 1000x1000 matrices.

```
size = 100

A = [[i for i in range(size)] for j in range(size)]
B = [[i for i in range(size)] for j in range(size)]

start = time.time()

result = [[0 for _ in range(size)] for _ in range(size)]

for i in range(size):
    for j in range(size):
        for k in range(size):
            result[i][j] += A[i][k] * B[k][j]

end = time.time()
print("Time taken using Python lists:", end - start)

A_np = np.array(A)
B_np = np.array(B)

start = time.time()
result_np = np.dot(A_np, B_np)
end = time.time()

print("Time taken using NumPy arrays:", end - start)
```

```
Time taken using Python lists: 0.22635197639465332
Time taken using NumPy arrays: 0.0012536048889160156
```